

Phase 1: Backend core — วิธีทำ และคำอธิบายโค้ด

ทำ backend ให้ครบทั้ง 2 ทาง: รับข้อมูลที่ engine push มา (Connector) และให้ portal อ่านออกไป (REST + SSE) โดยแยกจาก engine ตามกฎ §3 และ §4 ของ CLAUDE.md

ฉบับวันที่ 9 ตุลาคม 2026 · คู่กับ docs/rebuild-plan.pdf (หน้า 11–13)

ทดสอบกับ Zabbix 7.0.29 · TimescaleDB 2.28.3 (PG16) · Redis 7.4.5 · Python 3.12 (container) / 3.14 (local venv)

ผลการทดสอบในเอกสารนี้เป็นผลจริงจากเครื่องนี้ ไม่ใช่ตัวอย่างสมมติ

1. สรุปผล: Phase 1 ผ่านเงื่อนไขปิดครบ
2. ภาพรวม: ไฟล์ทั้งหมด และเส้นทางข้อมูล
3. เตรียมเครื่อง: dependency และ `backend/.env`
4. 1.1 Domain: models, enums, ports
5. 1.2 Connector schema
6. 1.3 Mapper, adapter, connector ingest
7. 1.4 Store ของเราเอง
8. 1.5 `api/hooks.py` : ตัวรับ push จริง
9. 1.6 Read services + API endpoints
10. 1.7 Multi-tenant และ auth
11. 1.8 Live updates: bus, SSE, cache
12. 1.9 Wiring และ test
13. ทดสอบแบบ end-to-end กับ engine จริง (Docker)
14. Decision ที่ปิดใน Phase นี้ (ADR 0006–0008)
15. ข้อจำกัดที่รู้อยู่ และงานต่อใน Phase 2

วิธีอ่าน: แต่ละชั้นมีหัวข้อเหมือนกันหมด คือ เป้าหมาย → ไฟล์ → อธิบายโค้ด → ตรวจสอบว่าผ่าน → ข้อผิดพลาดที่พบบ่อย โค้ดในเอกสารเป็น **ส่วนที่สำคัญ** ของแต่ละไฟล์ โค้ดเต็มอยู่ใน repo ตาม path ที่ระบุ คำสั่งทั้งหมดรันจาก **Git Bash ที่ root ของ repo** ยกเว้นที่เขียนว่า `cd backend`

1. สรุปผล: Phase 1 ผ่านเงื่อนไขปิดครบ

เงื่อนไขปิด phase จาก rebuild plan: "push probe เป็น 1 แล้ว problem ต้องโผล่ใน API ของเราและมาทาง SSE ภายในไม่กี่วินาที โดยไม่มีการเรียก history.get , pytest ผ่านทั้งหมด, check-boundary.sh ได้ OK"

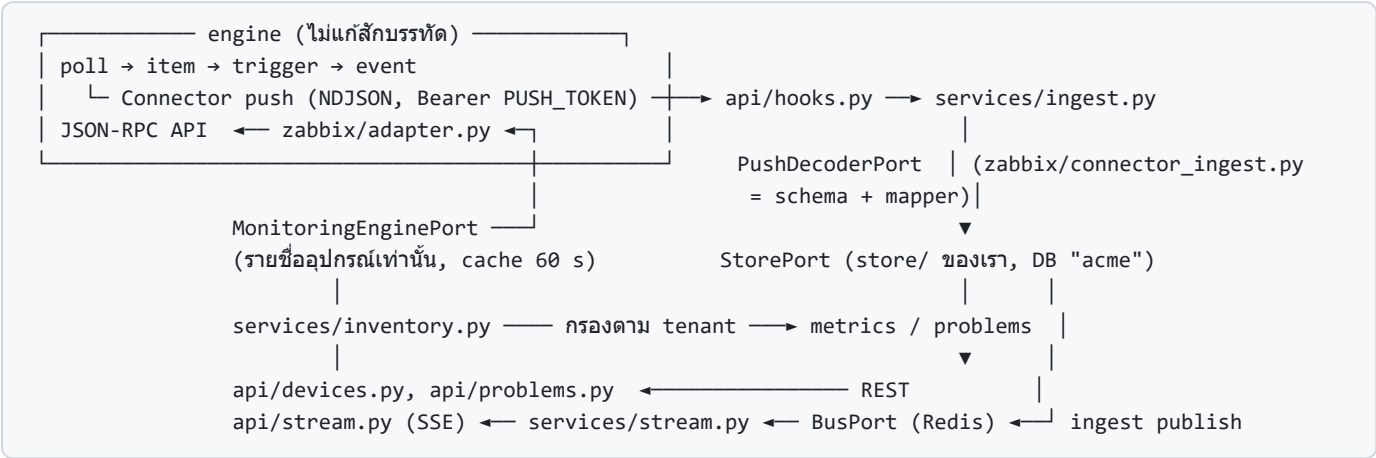
เงื่อนไข	ผลจริง (9 ต.ค. 2026)	ผ่าน
push probe = 1 → problem มาทาง SSE	push เวลา 18:22:48, SSE ได้ event: problem ref P22 เวลา 18:22:51 (3 วินาที)	✓
problem โผล่ใน GET /api/problems	[{"ref":"P22","severity":"warning","active":true,...}]	✓
push probe = 0 → resolved	SSE ได้ข้อความที่ 2 "active": false , /api/problems คืน []	✓
ไม่มี history.get	grep -rn "history\.get" backend/app → ไม่พบ; เรียก /metrics + /latest 20 ครั้ง → engine ได้ request 0 ครั้ง	✓
pytest ผ่านทั้งหมด	local: 78 passed , 8 skipped (Postgres) · ใน container: Postgres store 16 passed	✓
check-boundary.sh	check-boundary: OK (backend/app/domain backend/app/services backend/app/api backend/app/auth)	✓
store แยกจาก engine	psql -U acme_app -d zabbix → FATAL: permission denied for database "zabbix"	✓
ADR 0004: tenant token เห็นเฉพาะ host ของตัวเอง	ADR 0004 check, tenant lab: PASS (sees ['Zabbix server'], expected ['Zabbix server'])	✓

สิ่งที่ผมเปลี่ยนในระบบที่รันอยู่ (ความรู้):

- สร้าง role acme_app และ database acme, acme_test ใน Postgres ของ engine และ **REVOKE CONNECT ON DATABASE zabbix FROM PUBLIC**
- Connector id 1, 2 เปลี่ยนชื่อเป็น acme-values / acme-events และชี้ไปที่ http://backend:8000/hooks/... (เดิมชี้ไป sink) ส่วน id 3 (lab-sink-probes) ถูก **disable**
- สร้าง user group acme-lab-readers, user acme-lab-reader และ token acme-read ใน engine สำหรับ tenant ทดลอง lab
- เพิ่ม key ใหม่ใน backend/.env (ไฟล์นี้ gitignored): STORE_URL , ACME_DB_PASSWORD , PUSH_TOKEN , SESSION_SECRET , TOKEN_KEY , COOKIE_SECURE
- container ใหม่ 2 ตัว: NetWatch-backend , NetWatch-redis

2. ภาพรวม: ไฟล์ทั้งหมด และเส้นทางข้อมูล

เส้นทางข้อมูล



ไฟล์ที่สร้าง/แก้ไขใน Phase 1

ชั้น	ไฟล์	หน้าที่
1.1	backend/app/domain/enums.py, models.py, ports.py	ภาษาของเรา + สัญญา (Protocol) ทุกตัว
1.2	backend/app/zabbix/connector_schema.py backend/tests/fixtures/connector/*.ndjson	ตรวจรูปแบบ NDJSON จาก payload จริง
1.3	backend/app/zabbix/mapper.py, adapter.py, connector_ingest.py	ล่ามแปล engine ↔ domain
1.4	backend/app/store/sqlite_store.py, postgres_store.py , backend/app/crypto.py, ops/init-store.sh	store ของเรา + เข้ารหัส tenant token
1.5	backend/app/api/hooks.py, backend/app/services/ingest.py	รับ push → store → bus
1.6	backend/app/services/inventory.py, metrics.py, problems.py, views.py backend/app/api/devices.py, problems.py, deps.py	REST API ให้ portal
1.7	backend/app/auth/passwords.py, sessions.py, service.py, cli.py , backend/app/api/auth.py backend/app/zabbix/lab_tenant.py	login, user → tenant, tenant ทดลอง
1.8	backend/app/bus/memory.py, redis_bus.py, backend/app/cache.py , backend/app/services/stream.py, backend/app/api/stream.py	SSE ข้าม worker ผ่าน Redis
1.9	backend/app/container.py, main.py, config.py, backend/Dockerfile , backend/docker-compose.dev.yml, backend/tests/*	ประกอบทุกส่วน + test
ADR	docs/decisions/0006-*.md, 0007-*.md, 0008-*.md, CLAUDE.md §8	decision ใหม่ 3 เรื่อง

กฎที่ทุกไฟล์ต้องผ่าน: ใน domain/ services/ api/ auth/ ห้ามมีคำว่า zabbix, hostid, eventid, itemid, triggerid, clock, ns แม้แต่ใน comment ส่วนที่รู้จัก engine ได้มีแค่ backend/app/zabbix/** และ backend/app/container.py hook ของ Claude Code รัน ops/check-boundary.sh ทุกครั้งที่มีการเขียนไฟล์

3. เตรียมเครื่อง

Dependency (pip เท่านั้น ไม่ clone อะไร)

ไฟล์: backend/pyproject.toml เพิ่ม fastapi, uvicorn, redis, asyncpg, cryptography และ pytest (extra dev)

```
cd backend
.venv/Scripts/python -m pip install -e ".[dev]"
```

Package	ใช้ที่	ทำไม
fastapi + uvicorn	main.py, api/	web framework + ASGI server
asyncpg	store/postgres_store.py	driver Postgres แบบ async
redis	bus/redis_bus.py, cache.py	pub/sub ข้าม worker + cache
cryptography	crypto.py	Fernet เข้ารหัส tenant token ใน DB

SQLite, script (hash password) และ HMAC (session) ใช้ **stdlib** ไม่ต้องลงเพิ่ม

backend/.env : key ใหม่

```
# ดู backend/.env.example - สร้างค่าแต่ละตัวด้วยคำสั่งในคอมเมนต์
STORE_URL=sqlite:///./acme-store.db          # local; ใน container ใช้ Postgres (ตั้งโดย compose)
ACME_DB_PASSWORD=...                        # รหัสของ role acme_app
PUSH_TOKEN=...                             # python -c "import secrets; print(secrets.token_urlsafe(32))"
SESSION_SECRET=...                         # เหมือนกัน (อย่างน้อย 32 ตัวอักษร)
TOKEN_KEY=...                              # python -c "from cryptography.fernet import Fernet;
print(Fernet.generate_key().decode())"
COOKIE_SECURE=false # เฉพาะ local http
```

ข้อผิดพลาดที่พบบ่อย: ใช้ TOKEN_KEY ที่ไม่ได้มาจาก Fernet.generate_key() → ValueError: Fernet key must be 32 url-safe base64-encoded bytes · ห้ามเปลี่ยน TOKEN_KEY หลังบันทึก tenant แล้ว เพราะ token เดิมจะถอดรหัสไม่ได้ (ต้องบันทึก tenant ใหม่) · ENGINE_TOKEN (admin) ไม่จำเป็นสำหรับ backend ที่รันอยู่แล้ว compose จะตั้งให้เป็นค่าว่างใน container

4. ชั้น 1.1 — Domain: models, enums, ports

เป้าหมาย นิยามภาษาของ product เอง ไม่ผูกกับ engine

ไฟล์ backend/app/domain/enums.py, models.py, ports.py

enums.py

```
class Severity(IntEnum):
    INFO = 0
    WARNING = 1
    MINOR = 2
    MAJOR = 3
    CRITICAL = 4

class DeviceStatus(StrEnum):
    UP = "up"; DOWN = "down"; UNKNOWN = "unknown"; MAINTENANCE = "maintenance"; DISABLED = "disabled"
```

- **IntEnum** ทำให้เรียงลำดับได้: severity >= Severity.MAJOR ใช้กรองได้ตรงๆ
- ตัวเลขของเราไม่ต้องตรงกับของ engine (engine มี 0–5) การแปลงอยู่ใน mapper.py ที่เดียว
- **StrEnum**: ส่งออกเป็น JSON ได้ค่าอ่านรู้เรื่อง ("up") ไม่ใช่ตัวเลข

models.py

```
def _require_aware(field: str, value: datetime | None) -> None:
    if value is not None and value.tzinfo is None:
        raise ValueError(f"{field} must be timezone-aware, got naive {value!r}")

@dataclass(frozen=True, slots=True)
class MetricPoint:
    device_ref: str
    metric: str
    ts: datetime
    value: float

    def __post_init__(self) -> None:
        _require_aware("ts", self.ts)
```

- **@dataclass(frozen=True)**: แบบฟอร์มที่กรอกแล้วแก้ไม่ได้ จึงส่งข้าม worker หรือเก็บใน cache ได้ปลอดภัย
- **slots=True**: ใช้หน่วยความจำน้อยลง และกั้นการพิมพ์ชื่อ attribute ผิด
- **__post_init__**: รันทันทีหลังสร้าง object ถ้า datetime ไม่มี timezone ก็ error ตรงนั้นเลย ดีกว่าไปเจอทีหลังเป็นกราฟเลื่อน 7 ชั่วโมง
- device_ref เป็น str แบบ **opaque** คือ services ห้ามแกะความหมายของมัน
- มี Device, MetricPoint, Problem ตาม plan และเพิ่ม 3 ตัว: ProblemResolution (event "หายแล้ว" ที่ engine ส่งมาโดยไม่มีชื่อหรือ host), Tenant และ User (ใช้ใน 1.7)

ports.py

```
@runtime_checkable
class MonitoringEnginePort(Protocol):
    async def list_devices(self, token: str) -> list[Device]: ...
    async def get_device(self, ref: str, token: str) -> Device: ...

class PushDecoderPort(Protocol):
    def decode_values(self, body: bytes) -> list[MetricPoint]: ...
    def decode_events(self, body: bytes) -> list[Problem | ProblemResolution]: ...

class StorePort(Protocol): ...      # write_metrics, open_problem, resolve_problem, series, latest,
                                    # problems, tenants/users
class BusPort(Protocol): ...       # publish, subscribe
class CachePort(Protocol): ...     # get, set

class DeviceNotFound(LookupError): ...
class EngineUnavailable(RuntimeError): ...
```

- **Protocol** = "ประกาศรับสมัครงาน" คือ class ไหนมี method ตรงกันก็ใช้ได้ ไม่ต้อง inherit ส่วน `@runtime_checkable` ทำให้ test เช็คได้ด้วย `isinstance(adapter, MonitoringEnginePort)`
- **MonitoringEnginePort** ไม่มี method อ่าน history อันนี้จิงใจ ถ้าวันไหนอยากเพิ่ม `get_history` แปลว่ากำลังจะละเมิด §3
- **token** เป็น parameter ทุก method (ADR 0004) ทำให้ read path ใช้ token ของ tenant เสมอ
- `EngineUnavailable` มีไว้ให้ api/ ตอบ 502 ได้โดยไม่ต้องรู้จัก error class ของ engine
- port อื่นๆ (Store, Bus, Cache, PushDecoder) อยู่ไฟล์เดียวกัน ทำให้ services ฟัง domain/ports.py อย่างเดียว

ตรวจว่าผ่าน

```
bash ops/check-boundary.sh
# check-boundary: OK (backend/app/domain)
cd backend && .venv/Scripts/python -c "import app.domain.ports; print('ok')"
# ok
```

ข้อผิดพลาดที่พบบ่อย

- เขียนคำต้องห้ามใน **comment/docstring** เช่น "ไม่ใช่คู่ clock/ns" แค่นี้ก็ fail แล้ว เพราะ script ใช้ grep ทั้งไฟล์
- ใช้ `datetime.utcnow()` ซึ่งคืนค่า naive ให้ใช้ `datetime.now(UTC)`
- `ref: int` เพราะเห็นว่า ID ของ engine เป็นตัวเลข ให้ใช้ `str` เสมอ

5. ชั้น 1.2 — Connector schema

เป้าหมาย	แปลง NDJSON ที่ engine push มาเป็น object ที่ validate แล้ว โดยเขียนจาก payload จริง
ไฟล์	backend/app/zabbix/connector_schema.py, backend/tests/fixtures/connector/*.ndjson (copy จาก lab/ndjson-sink/captured/), backend/tests/test_connector_schema.py

โค้ดหลัก

```
class _Line(BaseModel):
    model_config = ConfigDict(extra="ignore", frozen=True)

class ValueLine(_Line):
    host: HostRef
    itemid: int
    name: str
    clock: int
    ns: int = 0
    value: int | float | str      # int ก่อน float: counter แบบ unsigned ไม่เสียความแม่นยำ
    type: int                    # value_type: 0 float, 1 char, 2 log, 3 unsigned, 4 text

class ProblemLine(_Line):
    value: Literal[1]
    eventid: int; clock: int; ns: int = 0
    name: str; severity: int; hosts: list[HostRef]

class RecoveryLine(_Line):
    value: Literal[0]
    eventid: int; p_eventid: int; clock: int; ns: int = 0

EventLine = Annotated[ProblemLine | RecoveryLine, Field(discriminator="value")]
```

- `extra="ignore"`: patch release อาจเพิ่ม field ใหม่ เราไม่ให้ระบบพังเพราะเรื่องนี้
- **ทุก field มาจากไฟล์ใน captured/** (ดู docs/connector-payload.md) ไม่มีตัวไหนมาจาก "เอกสารบอกว่า"
- `value: int | float | str`: ใน payload จริง ค่าของ item แบบ float มาเป็น JSON integer ได้ (11 จาก 23 บรรทัดเป็น 0 ไม่ใช่ 0.0) และ Pydantic โหมด smart จะเลือก type ที่ตรงที่สุด
- **`discriminator="value"`**: problem กับ recovery มีหน้าตาต่างกัน Pydantic ดูค่า `value` (1 หรือ 0) แล้วเลือก model ให้ถูกตัว ถ้าเป็นค่าอื่นจะ error แล้วบรรทัดนั้นถูกข้าม

```
def parse_ndjson(body: bytes, adapter: TypeAdapter[T], *, kind: str) -> list[T]:
    parsed: list[T] = []
    for number, raw in enumerate(body.splitlines(), start=1):
        if not raw.strip():
            continue
        try:
            parsed.append(adapter.validate_python(json.loads(raw)))
        except (ValueError, ValidationError) as exc:
            log.warning("connector %s line %d skipped: %s | %.200r", kind, number, ...)
    return parsed
```

- **parse ที่ละบรรทัด** ถ้าบรรทัดไหนเสียก็ log แล้วข้าม **ไม่ทิ้งทั้ง batch**
- `json.JSONDecodeError` เป็นลูกของ `ValueError` จึงจับได้ใน except เดียว
- `%.200r` ตัด log ไว้ที่ 200 ตัวอักษร กันบรรทัดยั๊ยหาม log

ตรวจว่าผ่าน

```
cd backend && .venv/Scripts/python -m pytest tests/test_connector_schema.py -q
# 8 passed
```

test ครอบ: parse fixture จริงครบทุกบรรทัด (values 20 ไฟล์ + probes), เห็น value_type 1–4, float ที่มาเป็น JSON integer, problem/recovery, บรรทัดเสียปนกับบรรทัดดี, field แปลกที่ไม่รู้จัก และ event value ที่ไม่รู้จัก

ข้อผิดพลาดที่พบบ่อย

- ตั้ง value: float แล้วเจอค่า text ทำให้ทั้งบรรทัด fail
- log type 2 มี field eventid ของบันทึก log (เช่น Windows event log) ซึ่งไม่เกี่ยวกับ event stream อย่าเอามาปนกัน (schema ไม่ประกาศ field นี้ จึงถูก ignore)
- เขียน schema จากเอกสารแทน payload จริง

6. ชั้น 1.3 — Mapper, adapter, connector ingest

เป้าหมาย จุดเดียวที่แปลระหว่างโลกของ engine กับ domain ของเรา

ไฟล์ backend/app/zabbix/mapper.py, adapter.py, connector_ingest.py

mapper.py: ref ที่ opaque

```
def device_ref(host: str) -> str:          return host
def host_from_device_ref(ref: str) -> str:  return ref
def problem_ref(eventid: int | str) -> str: return f"P{int(eventid)}"
def eventid_from_problem_ref(ref: str) -> int:
    if not ref.startswith("P") or not ref[1:].isdigit():
        raise ValueError(f"not a problem ref: {ref!r}")
    return int(ref[1:])
```

- ทำไม **device_ref** = ชื่อ **host** ไม่ใช่ **hostid** (ADR 0006): payload ของ Connector ไม่มี **hostid** มีแต่ **host.host** ถ้าใช้ **hostid** ต้องยิง API 1 ครั้งต่อบรรทัด ซึ่งช้าและอยู่บนเส้นทางที่ร้อนที่สุด
- ฟังก์ชันคู่ "สร้าง/แกะ" อยู่ที่นี่ทีเดียว ถ้าวันหนึ่งเปลี่ยนรูปแบบ **ref** ก็แก้แค่ไฟล์นี้

mapper.py: เวลา และ severity

```
def to_datetime(seconds: int, nanos: int = 0) -> datetime:
    return datetime.fromtimestamp(seconds, tz=UTC) + timedelta(microseconds=nanos // 1000)

_SEVERITY = {0: Severity.INFO, 1: Severity.INFO, 2: Severity.WARNING,
              3: Severity.MINOR, 4: Severity.MAJOR, 5: Severity.CRITICAL}
```

- engine ส่งเวลาเป็นคู่ (วินาที, นาโนวินาที) ส่วนเราเก็บเป็น **datetime** ตัวเดียวที่มี **tz=UTC** นาโนวินาทีถูกปัดเป็นไมโครวินาที เพราะ Python เก็บได้ละเอียดแค่นั้น
- severity ของ engine: 0 Not classified, 1 Information, 2 Warning, 3 Average, 4 High, 5 Disaster ค่าที่ไม่รู้จักจะกลายเป็น **WARNING** เพื่อไม่ให้ซ่อนปัญหา

mapper.py: device_from_api

```
def _status(host) -> DeviceStatus:
    if str(host.get("status")) == "1":          return DeviceStatus.DISABLED      # not monitored
    if str(host.get("maintenance_status")) == "1": return DeviceStatus.MAINTENANCE
    states = {str(i.get("available")) for i in host.get("interfaces", [])}
    if "1" in states: return DeviceStatus.UP      # available
    if "2" in states: return DeviceStatus.DOWN    # unavailable
    return DeviceStatus.UNKNOWN

def device_from_api(host) -> Device:
    tags = host.get("tags", [])
    return Device(ref=device_ref(host["host"]), name=host.get("name") or host["host"],
                  kind=_KIND.get(_tag(tags, "role"), DeviceKind.OTHER),
                  site=_tag(tags, "site"), status=_status(host))
```

- kind** และ **site** มาจาก **host tag** **role** และ **site** ซึ่ง provisioning (Phase 2) ต้องตั้งให้ทุก host ค่า **role** ที่รู้จักคือ **core**, **access**, **ap** และ **server**

- ใน Zabbix 7.0 ค่า availability อยู่ที่ **interface** ไม่ได้อยู่ที่ host (ขึ้นกับ version ถ้าเปลี่ยนเวอร์ชันให้ตรวจด้วย `host.get + selectInterfaces`)
- ไม่อ่าน `lastvalue` ตาม ADR 0005 เพราะค่าล่าสุดมาจาก store ของเรา

adapter.py

```
_HOST_QUERY = {"output": ["host", "name", "status", "maintenance_status"],
               "selectInterfaces": ["available"], "selectTags": ["tag", "value"], "sortfield": "name"}

class ZabbixAdapter:
    async def _hosts(self, params, token):
        try:
            return await self._client.call("host.get", params, token=token)
        except (ZabbixAPIError, httpx.HTTPError) as exc:
            raise EngineUnavailable(str(exc)) from exc

    async def list_devices(self, token: str) -> list[Device]:
        return [mapper.device_from_api(h) for h in await self._hosts(_HOST_QUERY, token)]
```

- implement `MonitoringEnginePort` โดยไม่ต้อง **inherit** เพราะ Protocol ดูแล้วมี method ครบ
- แปลง error ของ engine เป็น `EngineUnavailable` ฝั่ง api จึงไม่ต้องรู้จัก `ZabbixAPIError`
- class นี้ไม่ถือ token ของตัวเองเลย token มาจากผู้เรียกทุกครั้ง

connector_ingest.py

```
class ConnectorDecoder:
    def decode_values(self, body: bytes) -> list[MetricPoint]:
        points = (mapper.metric_from_value(line) for line in parse_values(body))
        return [p for p in points if p is not None]      # เก็บเฉพาะค่าตัวเลข (type 0, 3)
```

ทำหน้าที่ "ต่อสาย" schema (ตรวจรูปแบบ) เข้ากับ mapper (แปลภาษา) เพื่อให้ `api/hooks.py` ได้แต่ object ของ domain ส่วน value แบบ text, char และ log ยังไม่ถือเป็น metric

ตรวจว่าผ่าน

```
cd backend && .venv/Scripts/python -m pytest tests/test_mapper_adapter.py -q
# 24 passed
```

test ครอบ: ref แปลงไปแล้วกลับได้ (round-trip), ref เสียถูกปฏิเสธ, เวลาเป็น UTC aware, ตาราง severity, status ทุกกรณี, adapter ผ่าน `httpx.MockTransport` (ส่ง token ของผู้เรียกจริง และไม่ขอ `lastvalue`), decoder กับ fixture จริง

ข้อผิดพลาดที่พบบ่อย

- parse ref นอก mapper เช่น `ref[1:]` ใน `services` ซึ่งผิด §4
- ลืม `tz=UTC` ใน `fromtimestamp` ทำให้ได้เวลาเครื่อง (naive) แล้วถูก `__post_init__` ปฏิเสธ
- เปลี่ยนชื่อ host ใน engine แล้วกราฟเก่าหาย ไม่ใช่บั๊ก เป็นผลของ ADR 0006 (ชื่อ host = ID ตาราง)

7. ชั้น 1.4 — Store ของเราเอง

เป้าหมาย ที่เก็บ time-series, latest value, problem และบัญชี portal ที่ทุกกราฟและ tile อ่าน
ไฟล์ backend/app/store/sqlite_store.py (test/demo), postgres_store.py (appliance),
backend/app/crypto.py, ops/init-store.sh

Tables (เหมือนกันทั้ง 2 implementation)

Table	Key	ใช้ทำอะไร
metric_points	(device_ref, metric, ts)	กราฟ ใน Postgres เป็น hypertable
latest_values	(device_ref, metric)	tile "ค่าล่าสุด" (ADR 0005)
problems	ref	รายการปัญหา และสถานะ resolved
tenants	id	ลูกค้า + read token ที่เข้ารหัสแล้ว
users	username	user ของ portal + password hash

ใน plan ระบุ table devices ไว้ด้วย แต่ผมไม่ได้ทำ เพราะรายชื่ออุปกรณ์มาจาก engine port (มี cache) ถ้าเก็บซ้ำในเรามีโอกาสที่ข้อมูลสองที่จะไม่ตรงกัน

Upsert: เพราะ Connector อาจส่งซ้ำ

```
INSERT INTO metric_points VALUES ($1,$2,$3,$4)
ON CONFLICT (device_ref, metric, ts) DO UPDATE SET value = excluded.value;

INSERT INTO latest_values VALUES ($1,$2,$3,$4)
ON CONFLICT (device_ref, metric) DO UPDATE
SET ts = excluded.ts, value = excluded.value
WHERE excluded.ts >= latest_values.ts;    -- batch เก้าที่มาซ้ำ ห้ามทับค่าใหม่
```

- ส่ง batch เดิมซ้ำ 2-3 รอบ ข้อมูลก็ไม่ซ้ำ (เรียกว่า **idempotent**) เราจึงตั้ง Connector ให้ retry ได้ 3 ครั้งอย่างปลอดภัย
- WHERE excluded.ts >= latest_values.ts ทำให้ "ค่าล่าสุด" เดินหน้าอย่างเดียว
- problem ใช้ ON CONFLICT (ref) DO NOTHING ส่วน resolve ใช้ UPDATE ... WHERE resolved_at IS NULL

Downsampling ทำใน store

```
-- Postgres + TimescaleDB
SELECT time_bucket($1::interval, ts) AS bucket, avg(value) FROM metric_points
WHERE device_ref = $2 AND metric = $3 AND ts >= $4 AND ts < $5
GROUP BY bucket ORDER BY bucket;

-- SQLite (ผลเหมือนกัน)
SELECT CAST(ts / :step AS INTEGER) * :step AS bucket, AVG(value) ... GROUP BY bucket
```

Postgres: schema ตอน start + กันชนกัน

```
async with self._db.acquire() as conn, conn.transaction():
    await conn.execute("SELECT pg_advisory_xact_lock($1)", _SCHEMA_LOCK)
    await conn.execute(_SCHEMA)          # CREATE TABLE IF NOT EXISTS ... + create_hypertable(...)
    await conn.execute("SELECT add_retention_policy('metric_points', $1::interval, if_not_exists => TRUE)",
    ...)
```

บั๊กจริงที่เจอระหว่างทำ: พอ uvicorn รัน 2 worker ทั้งสองตัวรัน CREATE TABLE IF NOT EXISTS พร้อมกัน แล้ว Postgres ตอบ UniqueViolationError: duplicate key value violates unique constraint
"pg_type_typname_nsp_index" (คำว่า "IF NOT EXISTS" ไม่ได้กัน race) ทางแก้คือ **advisory lock** ภายใน transaction ซึ่งบังคับให้ทีละ worker บั๊กแบบนี้ unit test จับไม่ได้ ต้องเจอตอนรันจริง

SQLite: async โดยไม่บล็อก event loop

```
async def _run(self, fn):
    async with self._lock:
        return await asyncio.to_thread(fn, self._db)
```

sqlite3 ใน stdlib เป็นแบบ sync เราจึงโยนงานไปทำใน thread และใช้ lock กันไม่ให้ 2 thread ใช้ connection เดียวกันพร้อมกัน

crypto.py: tenant token เข้ารหัสใน DB

```
class TokenCipher:
    def __init__(self, key: str) -> None: self._fernet = Fernet(key.encode())
    def encrypt(self, plaintext: str) -> str: return self._fernet.encrypt(plaintext.encode()).decode()
    def decrypt(self, ciphertext: str) -> str: return self._fernet.decrypt(ciphertext.encode()).decode()
```

Fernet คือ AES-128-CBC + HMAC-SHA256 key มาจาก TOKEN_KEY ใน env ถ้า DB หลุดออกไป token ก็ยังอ่านไม่ได้ถ้าไม่มี key

ops/init-store.sh: สร้าง DB ของเราแยกจาก engine

```
SELECT 'CREATE ROLE acme_app LOGIN' WHERE NOT EXISTS (...) \gexec
ALTER ROLE acme_app WITH LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE PASSWORD : 'pw';
SELECT 'CREATE DATABASE acme OWNER acme_app' WHERE NOT EXISTS (...) \gexec
CREATE EXTENSION IF NOT EXISTS timescaledb;          -- ใน acme, โดย admin
REVOKE CONNECT ON DATABASE zabbix FROM PUBLIC;      -- ปิดประตู DB ของ engine
```

- \gexec คือผลลัพธ์ของ SELECT เป็นคำสั่ง ใช้ทำ "สร้างถ้ายังไม่มี" เพราะ Postgres ไม่มี CREATE DATABASE IF NOT EXISTS
- รหัสผ่านส่งผ่าน **psql variable บน stdin** (: 'pw') จึงไม่โผล่ใน process list
- **REVOKE CONNECT** คือหัวใจของ §3 เพราะโดยปกติ PUBLIC (ทุก role) connect ได้ทุก DB ถ้าไม่ revoke ตัว acme_app ก็เข้า DB ของ engine ได้

ตรวจว่าผ่าน

```
ACME_DB_PASSWORD=$(grep '^ACME_DB_PASSWORD=' backend/.env | cut -d= -f2- | tr -d '\r') bash ops/init-store.sh --test-db
# init-store: role acme_app, database(s) acme acme_test ready; CONNECT on zabbix revoked from PUBLIC

docker compose -f engine/docker-compose.yml exec -T db psql -U acme_app -d zabbix -c "select 1"
# FATAL: permission denied for database "zabbix"
# DETAIL: User does not have CONNECT privilege.

docker compose -f engine/docker-compose.yml exec -T db psql -U acme_app -d acme \
    -tAc "select proc_name, config from timescaledb_information.jobs where hypertable_name='metric_points'"
# policy_retention|{"drop_after": "30 days", "hypertable_id": 1}

cd backend && .venv/Scripts/python -m pytest tests/test_store.py -q      # SQLite: 8 passed, 8 skipped (Postgres)
# Postgres (รันใน container เพราะ DB ไม่เปิด port ออกนอก network):
PW=$(grep '^ACME_DB_PASSWORD=' backend/.env | cut -d= -f2- | tr -d '\r')
MSYS_NO_PATHCONV=1 docker compose -f backend/docker-compose.dev.yml run --rm --no-deps \
    -v "$(pwd -W)/backend/tests:/srv/tests:ro" \
    -e TEST_POSTGRES_DSN="postgresql://acme_app:${PW}@db:5432/acme_test" \
    backend python -m pytest tests/test_store.py -q -p no:cacheprovider
# 16 passed
```

ข้อผิดพลาดที่พบบ่อย

- ใช้ user zabbix กับ store ของเรา แบบนี้ไม่มีการแยกกันจริง
- ไม่มี unique key ทำให้ข้อมูลซ้ำทุกครั้ง Connector retry
- ลืม MSYS_NO_PATHCONV=1 และ pwd -W ใน Git Bash ทำให้ mount ได้โพลเดอร์ว่าง
- ชื่อ TEST_POSTGRES_DSN ไปที่ acme (DB จริง) เพราะ test จะ TRUNCATE ทุก table ต้องใช้ acme_test เท่านั้น

8. ชั้น 1.5 — api/hooks.py : ตัวรับ push จริง

เป้าหมาย แทน lab/ndjson-sink/sink.py ด้วย endpoint ของ backend

ไฟล์ backend/app/api/hooks.py , backend/app/services/ingest.py

hooks.py: ทำหน้าที่ขนส่งอย่างเดียว

```
def _check_token(request, expected):
    given = request.headers.get("Authorization", "")
    if not hmac.compare_digest(given.encode(), f"Bearer {expected}".encode()):
        raise HTTPException(401, "bad token")

@router.post("/values")
async def push_values(request: Request, container: ContainerDep) -> dict[str, int]:
    _check_token(request, container.settings.push_token)
    body = await request.body()
    try:
        stored = await container.ingest.values(body)
    except Exception as exc:
        raise HTTPException(503, "store unavailable") from exc
    return {"stored": stored}
```

- `hmac.compare_digest` ใช้เวลาเท่ากันไม่ว่า token จะผิดตรงตัวไหน คนนอกจึงเดา token จากเวลาตอบไม่ได้ (timing attack)
- ไม่มี schema และไม่มีคำว่า **engine** ในไฟล์นี้ ตัว decoder ถูกส่งเข้ามาผ่าน container ตามที่ plan กำหนด
- **status code** ออกแบบให้เข้ากับการ **retry** ของผู้ส่ง: 200 = เก็บแล้ว, 401 = config ผิด (retry ไปก็ไม่ช่วย), 503 = store มีปัญหา (retry ได้ เพราะเขียนแบบ idempotent)

services/ingest.py: เก็บก่อน แล้วค่อยประกาศ

```
async def values(self, body: bytes) -> int:
    points = self._decoder.decode_values(body)
    await self._store.write_metrics(points)          # 1) เก็บก่อน
    for device_ref, device_points in by_device.items():
        await self._bus.publish({"type": "metrics", "device_ref": device_ref, "points": [...]} ) # 2) ค่อยประกาศ

async def events(self, body: bytes) -> int:
    for item in self._decoder.decode_events(body):
        if isinstance(item, Problem):
            await self._store.open_problem(item); changed = item
        else:
            changed = await self._store.resolve_problem(item)
            if changed is None:
                log.warning("resolution for unknown problem %s ignored", item.problem_ref); continue
            await self._bus.publish({"type": "problem", "device_ref": changed.device_ref, "problem":
problem_view(changed)})
```

- **ลำดับสำคัญ:** ถ้าเก็บไม่สำเร็จ request จะ fail แล้วผู้ส่ง retry และจะไม่มีใครเห็นข้อมูลที่ไม่เคยถูกบันทึก
- **recovery** ไม่มีชื่อหรือ host มาด้วย จึงต้องให้ store หา problem เดิมก่อนแล้วคืนมา เราถึงรู้ว่าต้องส่ง SSE ไปที่อุปกรณ์ตัวไหน
- ถ้า recovery มาโดยไม่เคยเห็น problem (เช่น problem เกิดก่อนเราเริ่มฟัง) ระบบจะ log แล้วข้าม ไม่ crash

ย้าย Connector จาก sink มาที่ backend

```
ENGINE_TOKEN=$(grep '^ENGINE_TOKEN=' backend/.env | cut -d= -f2 | tr -d '\n')
PUSH_TOKEN=$(grep '^PUSH_TOKEN=' backend/.env | cut -d= -f2 | tr -d '\n')
api() { curl -s -X POST http://127.0.0.1:8080/api_jsonrpc.php -H 'Content-Type: application/json-rpc' \
-H "Authorization: Bearer $ENGINE_TOKEN" -d "$1"; echo; }
api '{"jsonrpc":"2.0","method":"connector.update","id":1,"params":['
{"connectorid":"1","name":"acme-
values","url":"http://backend:8000/hooks/values","token":"","max_attempts":3},
{"connectorid":"2","name":"acme-
events","url":"http://backend:8000/hooks/events","token":"","max_attempts":3},
{"connectorid":"3","status":0}]}'
docker compose -f engine/docker-compose.yml exec -T zabbix-server zabbix_server -R config_cache_reload
# '{"jsonrpc":"2.0","result":{"connectorids":["1","2","3"]},"id":1}
```

ขึ้นกับ version: connectorid 1, 2, 3 เป็นค่าบนเครื่องนี้ ให้เช็คของตัวเองก่อนด้วย connector.get ส่วน status 0 = disabled และ 1 = enabled (สำหรับ connector), max_attempts รับค่า 1-5 (ค่าที่อ้างอิง Zabbix 7.0.29)

ตรวจว่าผ่าน

```
docker logs NetWatch-backend 2>&1 | grep "values batch" | tail -2
# INFO app.api.hooks: values batch: 247 bytes, 1 points stored
# INFO app.api.hooks: values batch: 542 bytes, 2 points stored
cd backend && .venv/Scripts/python -m pytest tests/test_api.py -q -k "push or problem or retried or bad_lines"
```

ข้อผิดพลาดที่พบบ่อย

- ใช้ URL `http://localhost:8000` ซึ่งจากใน container ของ server แปลว่าตัว server เอง ต้องใช้ชื่อ service backend
- backend ไม่ได้อยู่ใน network `netwatch-engine` ทำให้ log ของ server ขึ้น `Couldn't resolve host`
- ตอบ 200 ทั้งที่ store เขียนไม่สำเร็จ ข้อมูลจะหายเสียๆ

9. ชั้น 1.6 — Read services + API endpoints

เป้าหมาย REST API ให้ portal: รายชื่ออุปกรณ์มาจาก engine port (มี cache) ส่วนตัวเลขทุกตัวมาจาก store
ไฟล์ backend/app/services/inventory.py, metrics.py, problems.py, views.py ,
backend/app/api/devices.py, problems.py, deps.py

Endpoints

Method + path	ข้อมูลมาจาก	หมายเหตุ
GET /api/devices	engine port (cache 60 s)	ใช้ token ของ tenant
GET /api/devices/{ref}	รายการข้างบน	404 ทั้งกรณี "ไม่มี" และ "ไม่ใช่ของคุณ"
GET /api/devices/{ref}/latest	store latest_values	tile ของอุปกรณ์
GET /api/devices/{ref}/metrics? metric=&from=&to=&step=	store metric_points	ไม่ส่ง metric = ทุก metric
GET /api/latest	store	tile ของหน้า overview
GET /api/problems?active=&device=&limit=	store problems	เรียงตาม active → severity → เวลา

inventory.py: รายชื่ออุปกรณ์ = ตัวกรองของทุกอย่าง

```
async def devices(self, tenant: Tenant) -> list[Device]:
    key = f"devices:{token_scope(tenant.engine_token)}"
    cached = await self._cache.get(key)
    if cached is not None:
        return [device_from_view(v) for v in cached]
    devices = await self._engine.list_devices(tenant.engine_token)
    await self._cache.set(key, [device_view(d) for d in devices], self._ttl)
    return devices

async def device(self, tenant, ref) -> Device:
    for d in await self.devices(tenant):
        if d.ref == ref: return d
    raise DeviceNotFound(ref)
```

- สิทธิ์ของ engine คำนวณแค่ทาง API ไม่ครอบคลุม store ของเรา ดังนั้นทุกการอ่านจาก store ต้องกรองด้วยชุด device_ref ของ tenant (ที่ได้จาก token ของ tenant) เสมอ
- cache key รวม hash ของ token (token_scope = sha256 ตัดเหลือ 16 ตัว) กันคำตอบของ tenant A ไปโผล่ที่ tenant B และ token จริงไม่อยู่ใน key
- ตอบ 404 เหมือนกันทั้ง "ไม่มี" และ "ไม่ใช่ของคุณ" คนนอกจะได้เดาไม่ได้ว่ามีอุปกรณ์อะไรอยู่บ้าง

metrics.py: จำกัดจำนวนจุด

```
MAX_POINTS = 2000; DEFAULT_POINTS = 300; MIN_STEP_SECONDS = 60
```

```
def choose_step(start, end, requested):  
    span = (end - start).total_seconds()  
    floor = max(MIN_STEP_SECONDS, math.ceil(span / MAX_POINTS))  
    if requested is None:  
        return max(floor, math.ceil(span / DEFAULT_POINTS))  
    return max(floor, requested)
```

กราฟไม่ต้องการจุดมากกว่าจำนวน pixel ถ้า client ขอ step เล็กเกินไป เราจะขยายให้เอง และตอบ step ที่ใช้จริงกลับไปด้วย
api/devices.py ปฏิเสธเวลาที่ไม่มี timezone (from=2026-10-07T00:00:00 → 400) ต้องส่งแบบ 2026-10-09T10:00:00+07:00

ตรวจว่าผ่าน (ตาม plan: เรียก /metrics แล้ว engine ต้องไม่ได้ request)

```
B=$(docker logs NetWatch-web 2>&1 | grep -c api_jsonrpc)  
for i in $(seq 10); do  
    curl -s -o /dev/null "http://127.0.0.1:8000/api/devices/Zabbix%20server/metrics" -H "Authorization: Bearer $SESSION"  
    curl -s -o /dev/null "http://127.0.0.1:8000/api/latest" -H "Authorization: Bearer $SESSION"  
done  
A=$(docker logs NetWatch-web 2>&1 | grep -c api_jsonrpc); echo $((A-B))  
# 0  
curl -s "http://127.0.0.1:8000/api/devices/Zabbix%20server/metrics?metric=Lab%20event%20probe&step=60" -H  
"Authorization: Bearer $SESSION"  
# {"device_ref":"Zabbix server","step":60,"series":{"Lab event probe":[{"ts":"2026-10-08T18:22:00+00:00","value":1.0},  
# {"ts":"2026-10-08T18:23:00+00:00","value":0.0}]}}
```

ข้อผิดพลาดที่พบบ่อย

- อ่าน store โดยไม่กรองตาม tenant ซึ่งเป็นช่องโหว่ข้ามลูกค้าที่ร้ายแรงที่สุดของระบบนี้
- ใส่ token ตรงๆ ใน cache key ทำให้ token โผล่ใน Redis
- ref ที่มีช่องว่าง (Zabbix server) ต้อง encode ใน URL เป็น %20 (portal ใช้ encodeURIComponent)

10. ชั้น 1.7 — Multi-tenant และ auth

เป้าหมาย user → tenant → read token ของ tenant โดย tenant A ต้องไม่เห็นข้อมูลของ B

ไฟล์ backend/app/auth/passwords.py, sessions.py, service.py, cli.py, backend/app/api/auth.py, deps.py, backend/app/zabbix/lab_tenant.py

Decision ที่ปิดในชั้นนี้ (ADR 0008): login ด้วย username + password แล้วได้ **session token** ที่ลงลายเซ็น **HMAC** ในรูป cookie แบบ HttpOnly (หรือส่งเป็น Bearer สำหรับ script) ยังไม่มี SSO

passwords.py: script จาก stdlib

```
def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.scrypt(password.encode(), salt=salt, n=2**14, r=8, p=1, dklen=32)
    return f"scrypt${n}${r}${p}${salt.hex()}${digest.hex()}"
```

- scrypt ตั้งใจให้ **ช้าและกินหน่วยความจำ** การเดารหัสจำนวนมากจึงแพง ส่วน salt ทำให้รหัสเดียวกันได้ hash ต่างกัน
- เก็บพารามิเตอร์ไว้ใน hash ทำให้หน้าเพิ่มความปลอดภัยได้โดย hash เก่ายังใช้ได้

sessions.py: token ลงลายเซ็น

```
payload = base64url(JSON {"u": username, "t": tenant_id, "exp": unix_seconds})
signature = base64url(HMAC-SHA256(SESSION_SECRET, payload))
token = payload + "." + signature

def verify(self, token):
    payload, sep, signature = token.partition(".")
    if not sep or not hmac.compare_digest(signature, self._sign(payload)): return None
    ... ถ้าหมดอายุ → None
```

- **stateless:** ทุก worker ตรวจ token ได้เองโดยไม่ต้องมี session store ร่วมกัน
- token **อ่านได้ แต่ปลอมไม่ได้** จึงไม่ใส่ความลับไว้ข้างใน (read token ของ tenant อยู่ใน DB)
- ใช้ cookie เพราะ EventSource (SSE ใน browser) **ตั้ง header เองไม่ได้** แต่ cookie ถูกส่งไปอัตโนมัติ

service.py

```
async def login(self, username, password) -> str | None:
    user = await self._store.get_user(username)
    if user is None:
        verify_password(password, _DUMMY_HASH) # ให้ใช้เวลาเท่ากับกรณีรหัสผิด
        return None
    ...

async def tenant_for(self, token) -> Tenant | None:
    session = self._signer.verify(token)
    user = await self._store.get_user(session.username) # อ่าน user ใหม่ทุกครั้ง
    if user is None or user.tenant_id != session.tenant_id: return None
    return await self._store.get_tenant(user.tenant_id)
```

- "ไม่มี user" กับ "รหัสผิด" ตอบเหมือนกันและใช้เวลาเท่ากัน คนนอกจึงเดาไม่ได้ว่ามี username อะไรบ้าง
- อ่าน user ใหม่ทุก request ทำให้ลบหรือย้าย user แล้วมีผลทันที ไม่ต้องรอ token หมดอายุ

สร้าง tenant ทดลอง และ user

provisioning/provision.py จะมาใน Phase 2 ระหว่างนี้ใช้ app/zabbix/lab_tenant.py ซึ่งสร้างใน engine ได้แค่ user group ที่อ่านได้อย่างเดียว ใน host group "Zabbix servers", user และ API token จากนั้นรัน check ตาม ADR 0004 แล้วบันทึก tenant lab ลง store (token ถูกเข้ารหัสและไม่ถูก print)

```
ENGINE_TOKEN=$(grep '^ENGINE_TOKEN=' backend/.env | cut -d= -f2 | tr -d '\r')
docker compose -f backend/docker-compose.dev.yml exec -T -e ENGINE_TOKEN="$ENGINE_TOKEN" backend python -m app.zabbix.lab_tenant
# ADR 0004 check, tenant lab: PASS (sees ['Zabbix server'], expected ['Zabbix server'])
# tenant 'lab' saved in our store (token encrypted; not printed)

docker compose -f backend/docker-compose.dev.yml exec backend python -m app.auth.cli add-user labuser lab
# password for labuser: (พิมพ์ซ่อน อย่างน้อย 12 ตัว)
# again:
# user labuser saved in tenant lab
```

ขึ้นกับ version: usergroup.create ใช้ hostgroup_rights (Zabbix 7.0) และ roleid "1" คือ role "User role" ที่ติดมากับระบบ ให้ตรวจด้วย role.get ก่อนใช้กับ instance อื่น

ตรวจว่าผ่าน

```
curl -s -X POST http://127.0.0.1:8000/api/auth/login -H 'Content-Type: application/json' \
-d '{"username":"labuser","password":"..."}' # {"token":"..."}
curl -s http://127.0.0.1:8000/api/auth/me -H "Authorization: Bearer $SESSION"
# {"tenant_id":"lab","tenant_name":"Lab tenant"}
curl -s http://127.0.0.1:8000/api/devices -H "Authorization: Bearer $SESSION"
# [{"ref":"Zabbix server","name":"Zabbix server","kind":"other","site":"","status":"down"}]
```

status: down ถูกต้องแล้ว เพราะ host "Zabbix server" มี agent interface ที่ติดต่อไม่ได้จริง (ใน lab ไม่มี agent รันอยู่) test test_tenants_see_only_their_own_devices และ test_tenant_cannot_read_another_tenants_data ยืนยันว่า tenant B อ่านอุปกรณ์ของ A ไม่ได้เลยทุกช่องทาง (404 หรือรายการว่าง)

ข้อผิดพลาดที่พบบ่อย

- ใช้ admin token ใน read path ซึ่งผิด ADR 0004 (container ของ backend ตั้ง ENGINE_TOKEN: "" ไว้กันพลาด)
- ส่ง password หรือ token เป็น argument บน command line ทำให้ค้างใน shell history (CLI ใช้ getpass แทน)
- บอก client ว่า "ไม่มี user นี้" ต่างจาก "รหัสผิด"

11. ชั้น 1.8 — Live updates: bus, SSE, cache

เป้าหมาย	เกิดปัญหาแล้วหน้าเว็บรู้ทันที แม้ request push กับ SSE จะอยู่คนละ uvicorn worker
ไฟล์	backend/app/bus/memory.py, redis_bus.py, backend/app/services/stream.py, backend/app/api/stream.py, backend/app/cache.py

ทำไมต้องมี Redis

worker #1 worker #2
POST /hooks/events → ingest → publish → Redis channel "acme:live" → subscribe → SSE → browser

bus แบบ memory ส่งได้แค่ใน worker เดียวกัน ถ้า push เข้า worker #1 แต่ browser ต่ออยู่กับ worker #2 ข้อความจะไม่ถึง Dockerfile จึงตั้ง `--workers 2` โดยตั้งใจ เพื่อให้การทดสอบพิสูจน์เส้นทางผ่าน Redis จริง

bus/memory.py: queue มีขนาดจำกัด

```
async def publish(self, message):
    for queue in self._subscribers:
        try: queue.put_nowait(message)
        except asyncio.QueueFull: log.warning("bus subscriber is full; message dropped")
```

browser ที่ค้าง (ไม่อ่าน) จะทำข้อความหาย แทนที่จะทำให้ memory โตไม่จำกัด เมื่อ reconnect แล้ว portal ค่อยอ่านสถานะใหม่จาก REST

services/stream.py: กรองตาม tenant + heartbeat

```
async with self._bus.subscribe() as messages:
    while True:
        if pending is None:
            pending = asyncio.ensure_future(anext(messages))
            done, _ = await asyncio.wait({pending}, timeout=self._heartbeat)
            if not done:
                yield HEARTBEAT; continue
            message, pending = pending.result(), None
            if message.get("device_ref") in refs:
                yield message
```

- bus มีข้อความของทุก **tenant** ตรงนี้คือจุดที่แต่ละ connection ได้เฉพาะของตัวเอง
- ถ้าไม่มีข้อความ 15 วินาทีจะส่ง heartbeat เพื่อกัน proxy ตัด connection ที่เสีย และให้ browser รู้เมื่อ connection ตาย
- ใช้ `asyncio.wait` กับ task ที่ค้างไว้ แทน `wait_for` เพราะ `wait_for` จะ cancel การอ่านทุกครั้งที่ timeout และอาจทำข้อความหาย
- รายชื่ออุปกรณ์ถูก refresh ทุก 60 วินาที อุปกรณ์ใหม่จึงเริ่ม stream ได้โดยไม่ต้อง reconnect

api/stream.py: รูปแบบ SSE

```
: connected

event: problem
data: {"type": "problem", "device_ref": "Zabbix server", "problem": {"ref": "P22", ..., "active": true}}

: heartbeat
```

บรรทัดที่ขึ้นต้นด้วย : คือ comment ซึ่ง EventSource จะไม่สนใจ header X-Accel-Buffering: no บอก nginx ใน Phase 4 ว่าไม่ต้อง buffer

ตรวจว่าผ่าน: เจื่อนไขปิด phase

```
# terminal 1: เปิด stream ค้างไว้
curl -N http://127.0.0.1:8080/api/stream -H "Authorization: Bearer $SESSION"

# terminal 2: push probe = 1 (item lab.event.probe, id 50740 บนเครื่องนี้)
curl -s -X POST http://127.0.0.1:8080/api_jsonrpc.php -H 'Content-Type: application/json-rpc' \
-H "Authorization: Bearer $ENGINE_TOKEN" \
-d '{"jsonrpc": "2.0", "method": "history.push", "params": [{"itemid": "50740", "value": "1"}], "id": 4}'

# ผลจริง: push 18:22:48 → terminal 1 ได้ภายใน 3 วินาที
# event: problem
# data: {"type": "problem", ..., "ref": "P22", "severity": "warning", "title": "Lab: event probe is 1",
"active": true}
# push value "0" → ได้ข้อความที่สอง "active": false, "resolved_at": "2026-10-08T18:23:01.691172+00:00"
```

history.push ตรงนี้คือคำสั่งที่ เราใช้ ส่งค่าเข้า engine เพื่อจำลองเหตุการณ์ ไม่ใช่การอ่าน history และไม่มีอยู่ในโค้ดของ backend

ข้อผิดพลาดที่พบบ่อย

- ใช้ memory bus กับหลาย worker แล้ว SSE ได้ข้อความบ้างไม่ได้บ้าง (ขึ้นกับว่า request ตกไป worker ไหน)
- สิมกรอง tenant ใน stream ทำให้ลูกค้าเห็น event ของลูกค้าอื่น
- proxy buffer SSE ทำให้ข้อความมาเป็นก้อนทีละหลายนาที

12. ชั้น 1.9 — Wiring และ test

เป้าหมาย	ต่อทุกกล่องเข้าด้วยกันในจุดเดียว เปิดเซิร์ฟเวอร์ และมี test ครอบคลุมชั้น
ไฟล์	backend/app/container.py, main.py, config.py, backend/Dockerfile, backend/docker-compose.dev.yml, backend/tests/*

container.py: จุดเดียวที่เลือกของจริง

```
def build_container(settings, *, engine=None, store=None) -> Container:
    client = None
    if engine is None:
        client = ZabbixClient(settings.engine_url)
        engine = ZabbixAdapter(client)
    store = store or build_store(settings) # sqlite:/// หรือ postgresql://
    bus, cache = (RedisBus(url), RedisCache(url)) if settings.redis_url else (MemoryBus(), MemoryCache())
    ...
    return Container(..., ingest=IngestService(decoder, store, bus), inventory=..., metrics=..., auth=...)
```

- เป็น 1 ใน 2 ที่ได้รับยกเว้นจากกฎ boundary เพราะ มันต้องเลือก class จริงมาเสียบเข้ากับ port
- ส่วนอื่นรับ Container ไปใช้ โดยไม่มีใคร import implementation เองเลย
- test ส่ง engine=FakeEngine() และ store=SqliteStore(...) เข้ามาแทนได้ โดยที่ services, api และ auth ยังเป็นของจริงทั้งหมด

main.py

- อ่าน settings ตอน **start** (lifespan) ไม่ใช่ตอน import ทำให้ test สร้าง app รอบ container ของตัวเองได้
- DeviceNotFound → 404, EngineUnavailable → 502 พร้อมข้อความ "monitoring backend unavailable" (ไม่บอกชื่อ engine)
- serve portal จาก FRONTEND_DIST ถ้ามี (Phase 3) โดย path ที่ไม่ใช่ไฟล์จะได้ index.html (SPA) และกัน path traversal ด้วย is_relative_to

Dockerfile + docker-compose.dev.yml

- base image python:3.12.11-slim (official) ลง dependency ก่อน copy โค้ด แก้โค้ดแล้ว build ใหม่จึงไม่ต้องลง dependency ซ้ำ
- รันเป็น user acme ไม่ใช่ root
- compose เข้าร่วม network netwatch-engine แบบ external backend จึงเรียก zabbix-web:8080 และ db:5432 ได้ และ Connector เรียก backend:8000 ได้
- เปิด port แค่ 127.0.0.1:8000 และ Redis ไม่เปิด port เลย

Test ทั้งหมด

ไฟล์	จำนวน	ครอบคลุม
test_connector_schema.py	8	1.2: payload จริง, บรรทัดเสีย, field ใหม่
test_mapper_adapter.py	24	1.3: ref, เวลา, severity, status, adapter (MockTransport), decoder
test_store.py	8 × 2	1.4: contract เดียวกันกับ SQLite และ Postgres
test_api.py	16	1.5–1.7: HTTP ทั้งเส้น, tenant isolation, cache, 502
test_stream_auth_domain.py	9	1.1, 1.7, 1.8: naive datetime, script, session, stream filter, heartbeat
test_boundary.py	13	\$4 ใน Python (ใช้ใน CI ที่ไม่มี bash) + ตัวอย่างที่ต้องจับได้/ต้องไม่จับ

```
cd backend && .venv/Scripts/python -m pytest -q
# 78 passed, 8 skipped          (8 skipped = Postgres; รันใน container ได้ 16 passed)
bash ops/check-boundary.sh
# check-boundary: OK (backend/app/domain backend/app/services backend/app/api backend/app/auth)
```

pytest จะขึ้น warning `StarletteDeprecationWarning: Using httpx with starlette.testclient is deprecated` ซึ่งมาจาก Starlette 1.7 ไม่ใช่โค้ดเรา test ยังผ่าน ค่อยย้ายไปใช้ client ตัวใหม่ที่หลัง

13. ทดสอบแบบ end-to-end กับ engine จริง

ลำดับทั้งหมดที่ใช้บนเครื่องนี้ ทำซ้ำได้ตั้งแต่ต้น:

```
# 0) engine ต้อง healthy
docker compose -f engine/docker-compose.yml ps

# 1) store ของเรา (ครั้งแรกครั้งเดียว)
ACME_DB_PASSWORD=$(grep '^ACME_DB_PASSWORD=' backend/.env | cut -d= -f2 | tr -d '\r') bash ops/init-store.sh --test-db

# 2) backend + Redis
docker compose -f backend/docker-compose.dev.yml up -d --build
curl -s http://127.0.0.1:8000/healthz # {"status":"ok"}
docker logs NetWatch-backend 2>&1 | grep "startup complete" # ต้องขึ้น 2 บรรทัด (2 workers)

# 3) tenant ทดลอง + user
ENGINE_TOKEN=$(grep '^ENGINE_TOKEN=' backend/.env | cut -d= -f2 | tr -d '\r')
docker compose -f backend/docker-compose.dev.yml exec -T -e ENGINE_TOKEN="$ENGINE_TOKEN" backend python -m app.zabbix.lab_tenant
docker compose -f backend/docker-compose.dev.yml exec backend python -m app.auth.cli add-user labuser lab

# 4) Connector → backend (ดูหัวข้อ 1.5)
# 5) login → SSE → push probe 1/0 (ดูหัวข้อ 1.8)
# 6) /metrics ไม่แตะ engine (ดูหัวข้อ 1.6)
```

ถ้าจะหยุด/เริ่มใหม่

```
docker compose -f backend/docker-compose.dev.yml down # หยุด backend + Redis (ข้อมูลใน acme ยังอยู่)
docker compose -f backend/docker-compose.dev.yml up -d # เริ่มใหม่
```

ระหว่างที่ backend หยุด Connector จะ retry แล้วทิ้งข้อมูลช่วงนั้น (max_attempts 3) ซึ่งยอมรับได้ใน dev ส่วนใน appliance ให้ดูคิวด้วย internal item zabbix[connector_queue] (Phase 4) ถ้าจะกลับไปใช้ sink ใน lab ให้ connector.update URL กลับไปที่ netwatch-sink

14. Decision ที่ปิดใน Phase นี้

ADR	เรื่อง	ตัดสินใจ	ผลที่ต้องรู้
0006	ช่วงใน device_ref	ชื่อ host เชิงเทคนิค (host.host)	payload ไม่มี hostid · เปลี่ยนชื่อ host = อุปกรณ์ใหม่ ให้ถือชื่อ host เป็น ID ถาวร
0007	store อยู่ที่ไหน + เก็บนานเท่าไร	DB acme + role acme_app ใน PG instance เดียวกัน, REVOKE CONNECT, ข้อมูลดิบเก็บ 30 วัน	ต้องยืนยัน retention กับลูกค้าก่อน Phase 4 · ย้ายไป container แยกได้ด้วย pg_dump
0008	วิธี login	password (scrypt) → token ลงลายเซ็น HMAC ใน cookie HttpOnly (หรือ Bearer)	ยังไม่มี SSO · rate limit ไปหาที่ reverse proxy ใน Phase 4

ทั้ง 3 เรื่องถูกเพิ่มลงในตาราง CLAUDE.md §8 แล้ว ถ้าไม่เห็นด้วยข้อไหน ให้แก้ ADR ก่อน แล้วค่อยแก้โค้ด (ส่วนที่ต้องแก้มีบอกไว้ในแต่ละ ADR)

15. ข้อจำกัดที่รู้ๆ และงานต่อใน Phase 2

เรื่อง	สถานะตอนนี้	ทำเมื่อไร
ชื่อ metric	ใช้ ชื่อ item จาก payload (ใน payload ไม่มี item key และ units)	Phase 2: ดึง key/units จาก <code>item.get</code> แล้ว cache ตาม itemid ใน mapper
counter 64-bit	เก็บเป็น DOUBLE ค่าที่เกิน 2^{53} จะเสียความแม่นยำที่หลักทศนิยม ยังไม่ได้เห็นค่าจริงใน lab	2.4: ตรวจสอบกับ <code>ifHCInOctets</code> จริง (ปกติ throughput ส่งเป็น bps แบบ float ซึ่งไม่มีปัญหานี้)
value แบบ text/log	ยังไม่เก็บ (ไม่ใช่ metric)	เมื่อมี use case เช่นแสดง firmware version
tenant + user	ใช้ <code>lab_tenant.py</code> + <code>app.auth.cli</code>	2.3: <code>provisioning/provision.py</code> ทำแทนแบบ idempotent พร้อม check ADR 0004 ทุกรอบ
host tag role , site	host ใน lab ยังไม่มี tag จึงขึ้นเป็น <code>kind: other</code> และ <code>site: ""</code>	2.3: provisioning ใส่ tag ตาม <code>inventory.csv</code>
problem ที่ข้ามหลาย host	เก็บไว้ได้ host แรก	ปรับถ้ามี trigger แบบหลาย host จริง
Connector ตอน backend สัม	retry 3 ครั้งแล้วทิ้ง	Phase 4: health check + เผื่อคิว

Phase 1 ปิดแล้ว ขั้นตอนถัดไปตาม rebuild plan คือ **Phase 2: Provisioning และอุปกรณ์จริง** เริ่มจาก 2.1 export template ต้นฉบับ ส่วน Phase 3 (frontend) เริ่มคู่ขนานได้เลย เพราะ read API ของ 1.6 คงที่แล้ว